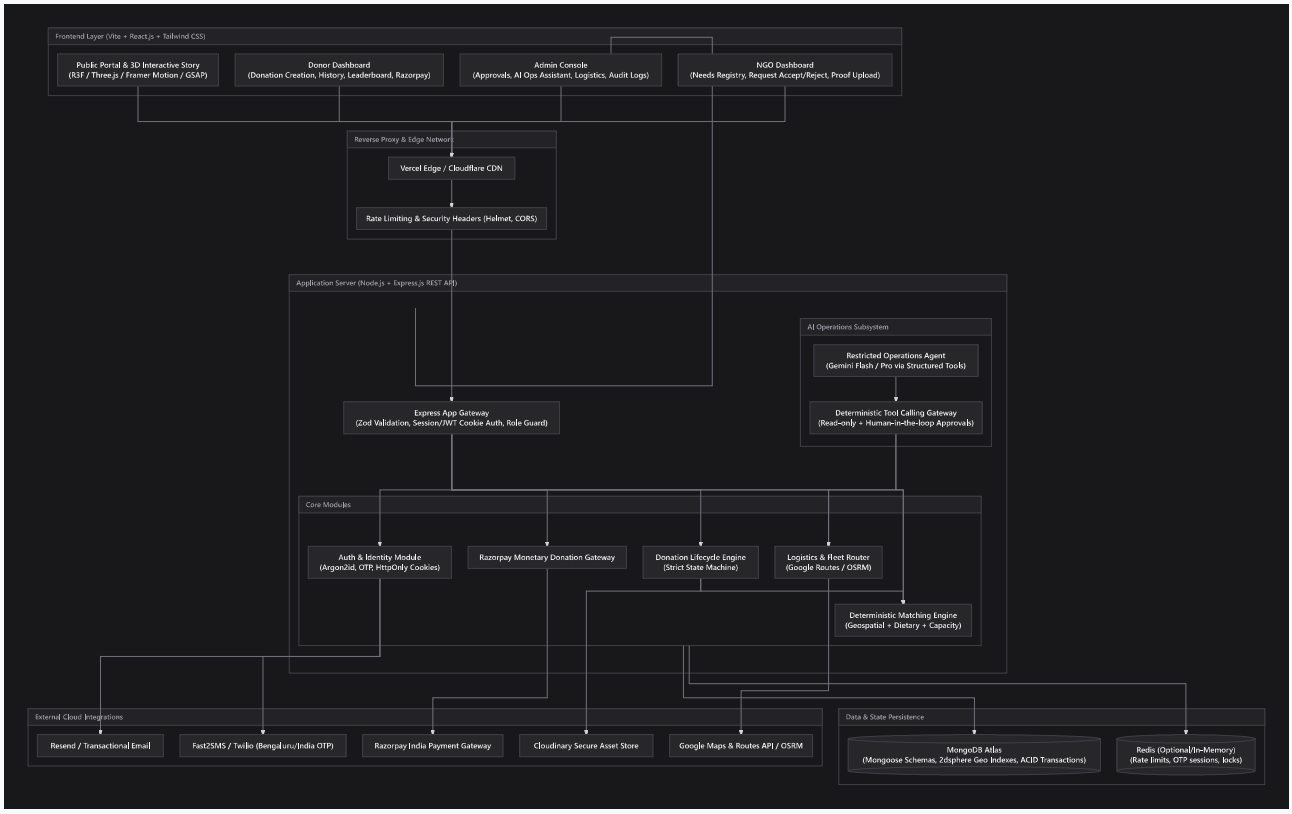
FOODBRIDGE — System Architecture & Implementation Plan

Tagline: *Rescue Food. Restore Hope.*
Initial Region: Bengaluru, Karnataka, India
Document Version: 1.0.0-PROPOSAL
Date: November 4, 2026

1. System Architecture Diagram



2. Directory Structure

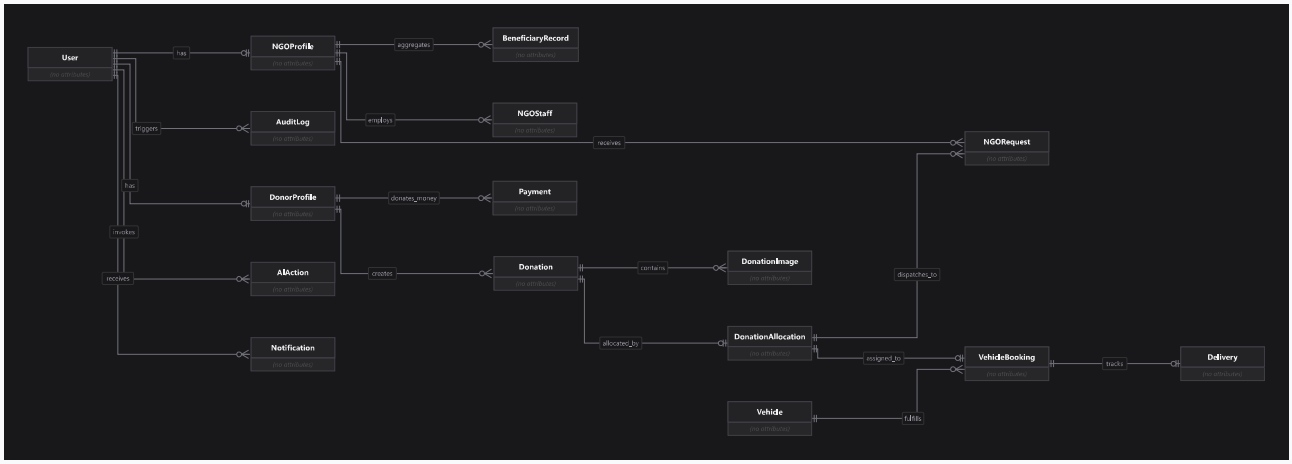
```
text
FoodBridge/
├── .github/
│   └── workflows/
│       ├── ci.yml                # Lint, Unit & Integration Tests, Semgrep
│       └── deploy.yml            # Automated staging/prod deploy
├── docs/
│   ├── ARCHITECTURE.md           # System topology, module bounds
│   ├── DATABASE_DESIGN.md        # Full Mongoose schemas, data dictionaries
│   ├── API_DOCUMENTATION.md      # OpenAPI 3.1 REST specs
│   ├── SECURITY.md               # OWASP Top 10 mitigation matrix, threat model
│   ├── AI_WORKFLOWS.md           # Tool schemas, prompt guardrails, human approvals
│   └── DEPLOYMENT.md             # Vercel & Render production runbook
├── shared/
│   ├── package.json
│   └── schemas/                  # Isomorphic Zod schemas shared by client & server
│       ├── index.ts
│       ├── auth.schema.ts
│       ├── donation.schema.ts
│       ├── ngo.schema.ts
│       └── logistics.schema.ts
├── client/
│   ├── index.html
│   ├── vite.config.js
│   ├── tailwind.config.js
│   ├── package.json
│   ├── public/
│   │   ├── models/               # 3D assets (.glb / .gltf optimized with draco)
│   │   └── textures/
│   └── src/
│       ├── components/           # Atomic reusable UI components (Button, Modal, etc.)
│       ├── pages/                # Route pages (Home, About, Login, Register)
│       ├── layouts/              # AdminLayout, DonorLayout, NGOLayout, RootLayout
│       ├── features/
│       │   ├── auth/             # Login, Register, OTP verification, Password Reset
│       │   ├── admin/            # Verification queue, Allocation desk, Fleet tracker
│       │   ├── donor/            # Post donation, History, Impact scorecard, Razorpay
│       │   ├── ngo/              # Capacity manager, Live requests, Receipt uploads
│       │   ├── donations/        # Shared donation status cards, timeline widgets
│       │   ├── logistics/        # Interactive route map, driver dispatch cards
│       │   └── ai-assistant/     # Admin AI copilot drawer with tool output previews
│       ├── hooks/                # Custom React hooks (useAuth, useGeolocation, useSocket)
│       ├── services/             # Axios / Fetch client wrappers with interceptors
│       ├── context/              # AuthContext, NotificationContext, ThemeContext
│       ├── three/                # 3D Canvas scenes (FoodJourneyScene, GlobeMesh)
│       ├── utils/                # Formatters, date math, coordinate converters
```

```
│       └── App.jsx
├── server/
│   ├── package.json
│   ├── .env.example
│   └── src/
│       ├── app.js                # Express app initialization & middleware stack
│       ├── server.js             # HTTP server & graceful shutdown handler
│       ├── config/               # DB connection, Cloudinary, Razorpay, Environment
│       ├── models/               # 18 Mongoose Schema definitions
│       ├── controllers/          # HTTP Request Handlers
│       ├── routes/               # Express API Route definitions
│       ├── services/             # Business Logic (DonationService, MatchingService, etc.)
│       ├── middleware/           # Auth guard, Role guard, Error handler, Rate limiters
│       ├── validators/           # Server-side Zod validation middleware
│       ├── utils/                # Crypto, Logger (Pino), Token helpers
│       ├── integrations/         # Resend, SMS OTP, Razorpay, Google Maps / OSRM
│       ├── ai/                   # Gemini LLM Provider, Tool declarations, Guardrails
│       ├── jobs/                 # Expiry cron check, OTP cleanup, Daily leaderboard
│       └── tests/                # Unit (Vitest), API (Supertest), Fixtures
├── .gitignore
├── README.md
└── package.json                  # Monorepo root workspace configuration (npm / pnpm)
```

> **Design Decision Note on `shared/`:**
> Having `shared/schemas/` ensures DRY (Don't Repeat Yourself) validation. The same Zod schemas validate user inputs client-side (instant feedback before network call) and server-side (zero-trust security barrier).

3. Database Entity Relationship & Schema Design (Mongoose)

3.1 Entity Relationship Diagram



3.2 Schema Specifications

Below are the 18 Mongoose schema specifications detailing types, validations, relationships, indexes, and access rules:

1. **`User`**
   - Fields: `_id`, `name`, `email` (unique, sparse), `phone` (unique, indexed), `passwordHash`, `role` (`'ADMIN' | 'DONOR' | 'NGO_MANAGER'`), `isEmailVerified` (bool), `isPhoneVerified` (bool), `status` (`'PENDING' | 'ACTIVE' | 'SUSPENDED'`), `lastLoginAt`, `refreshTokenHash`.
   - Indexes: `{ email: 1 }`, `{ phone: 1 }`, `{ role: 1 }`.
   - Security: `passwordHash` and `refreshTokenHash` explicitly set to `select: false`.
2. **`DonorProfile`**
   - Fields: `_id`, `userId` (`Ref: User`, unique), `donorType` (`'INDIVIDUAL' | 'RESTAURANT' | 'HOTEL' | 'MARRIAGE_HALL' | 'CORPORATE'`), `organizationName`, `fssaiLicenseNumber` (optional), `address` (`{ line1, area, city, pincode }`), `location` (`{ type: "Point", coordinates: [lng, lat] }`), `tier` (`'BRONZE' | 'SILVER' | 'GOLD' | 'PLATINUM'`), `totalMealsDonated` (num), `monetaryDonationTotal` (num).
   - Indexes: `{ location: "2dsphere" }`, `{ userId: 1 }`.
3. **`NGOProfile`**
   - Fields: `_id`, `userId` (`Ref: User`, unique), `ngoName`, `registrationNumber` (Darpan / 12A / 80G), `verificationStatus` (`'UNVERIFIED' | 'PENDING_REVIEW' | 'VERIFIED' | 'REJECTED'`), `verifiedBy` (`Ref: User`), `verifiedAt`, `rejectionReason`, `address` (`{ line1, area, city, pincode }`), `location` (`{ type: "Point", coordinates: [lng, lat] }`), `operatingHours` (`{ open: String, close: String }`), `storageFacilities` (`{ hasRefrigeration: Boolean, maxCapacityKg: Number }`), `foodPreferences` (`['VEG', 'NON_VEG', 'RAW_GRAINS', 'PACKAGED', 'COOKED']`).
   - Indexes: `{ location: "2dsphere" }`, `{ verificationStatus: 1 }`, `{ "foodPreferences": 1 }`.
4. **`NGOStaff`**
   - Fields: `_id`, `ngoId` (`Ref: NGOProfile`), `name`, `phone`, `role` (`'DIRECTOR' | 'VOLUNTEER_LEAD' | 'LOGISTICS_COORDINATOR'`), `isActive` (bool).
   - Indexes: `{ ngoId: 1 }`.
5. **`BeneficiaryRecord`** (Privacy-Preserving Aggregate Only)
   - Fields: `_id`, `ngoId` (`Ref: NGOProfile`), `dateRecorded` (Date), `targetGroup` (`'CHILDREN' | 'ELDERLY' | 'HOMELESS' | 'SLUM_COMMUNITY' | 'MIGRANT_WORKERS'`), `headcount` (Number, min: 1), `dietaryRequirements` (`['VEG_ONLY', 'HIGH_PROTEIN', 'SOFT_FOOD', 'NO_RESTRICTION']`), `mealTimePreference` (`['BREAKFAST', 'LUNCH', 'DINNER']`).
   - Rule: Strictly prohibits names, Aadhaar numbers, or biometric data.
   - Indexes: `{ ngoId: 1, dateRecorded: -1 }`.

6. **`Donation`**
   - Fields: `_id`, `donationCode` (String, unique format `FBD-YYYYMMDD-XXXX`), `donorId` (`Ref: DonorProfile`), `foodCategory` (`'COOKED_MEALS' | 'RAW_RATION' | 'PACKAGED_FOOD' | 'BAKERY_DAIRY'`), `dietaryType` (`'VEG' | 'NON_VEG' | 'VEGAN'`), `foodItems` (`[{ name: String, quantity: Number, unit: 'KG'|'PORTIONS'|'PACKETS', preparedAt: Date }]`), `perishable` (Boolean), `consumptionDeadline` (Date, required), `storageCondition` (`'AMBIENT' | 'REFRIGERATED' | 'HEATED'`), `pickupAddress` (`{ line1, area, city, pincode, contactPerson, contactPhone }`), `pickupLocation` (`{ type: "Point", coordinates: [lng, lat] }`), `status` (Enum - see Section 6), `rejectionReason` (String), `cancellationReason` (String).
   - Indexes: `{ pickupLocation: "2dsphere" }`, `{ status: 1 }`, `{ consumptionDeadline: 1 }`, `{ donorId: 1 }`.
7. **`DonationImage`**
   - Fields: `_id`, `donationId` (`Ref: Donation`), `cloudinaryPublicId`, `secureUrl`, `imageHash` (perceptual hash for duplicate/fraud detection), `uploadedBy` (`Ref: User`), `createdAt`.
   - Indexes: `{ donationId: 1 }`.
8. **`DonationAllocation`**
   - Fields: `_id`, `donationId` (`Ref: Donation`, unique), `allocatedByAdminId` (`Ref: User`), `selectedNgoId` (`Ref: NGOProfile`), `aiRecommendationScore` (Number, 0-100), `allocationNotes` (String), `allocatedAt` (Date), `status` (`'PROPOSED' | 'APPROVED' | 'ACCEPTED_BY_NGO' | 'REJECTED_BY_NGO' | 'SUPERSEDED'`).
   - Indexes: `{ donationId: 1 }`, `{ selectedNgoId: 1 }`.
9. **`NGORequest`**
   - Fields: `_id`, `allocationId` (`Ref: DonationAllocation`), `ngoId` (`Ref: NGOProfile`), `status` (`'PENDING' | 'ACCEPTED' | 'DECLINED' | 'TIMED_OUT'`), `responseDeadline` (Date, e.g. 15 minutes for perishable cooked food), `declinedReason` (String), `respondedAt` (Date).
   - Indexes: `{ allocationId: 1 }`, `{ ngoId: 1, status: 1 }`.
10. **`Vehicle`**
    - Fields: `_id`, `providerType` (`'INTERNAL_NGO' | 'AUTHORIZED_PARTNER' | 'VOLUNTEER'`), `vehicleType` (`'TWO_WHEELER' | 'AUTO_CARGO' | 'TEMPO_VAN' | 'REFRIGERATED_TRUCK'`), `registrationNumber` (String, unique), `driverName`, `driverPhone`, `capacityKg` (Number), `hasCooling` (Boolean), `isActive` (Boolean), `currentLocation` (`{ type: "Point", coordinates: [lng, lat] }`).
    - Indexes: `{ currentLocation: "2dsphere" }`, `{ isActive: 1 }`.
11. **`VehicleBooking`**
    - Fields: `_id`, `allocationId` (`Ref: DonationAllocation`), `vehicleId` (`Ref: Vehicle`), `bookingReference` (String, unique), `bookedBy` (`Ref: User`), `pickupEta` (Date), `dropoffEta` (Date),

`status` (`'ASSIGNED' | 'DISPATCHED' | 'ARRIVED_PICKUP' | 'IN_TRANSIT' | 'ARRIVED_DROPOFF' | 'COMPLETED' | 'CANCELLED'`).

- Indexes: `{ allocationId: 1 }`, `{ vehicleId: 1 }`, `{ status: 1 }`.

12. **`Delivery`**
    - Fields: `_id`, `bookingId` (`Ref: VehicleBooking`), `donationId` (`Ref: Donation`), `pickupOtp` (String, 6-digit hashed), `deliveryOtp` (String, 6-digit hashed), `pickupConfirmedAt` (Date), `deliveredAt` (Date), `receivedQuantity` (Number), `proofOfDeliveryImageUrl` (String), `ngoSignatoryName` (String), `temperatureAtDelivery` (Number, optional for cold-chain).
    - Indexes: `{ bookingId: 1 }`, `{ donationId: 1 }`.
13. **`Payment`** (Monetary Donations)
    - Fields: `_id`, `donorId` (`Ref: User`), `razorpayOrderId` (unique), `razorpayPaymentId` (unique, sparse), `razorpaySignature`, `amountInPaise` (Number), `currency` (Default: `'INR'`), `status` (`'CREATED' | 'CAPTURED' | 'FAILED' | 'REFUNDED'`), `taxExemption80GRequested` (Boolean), `panNumber` (Encrypted / masked).
    - Indexes: `{ donorId: 1 }`, `{ razorpayOrderId: 1 }`.
14. **`Notification`**
    - Fields: `_id`, `recipientId` (`Ref: User`), `type` (`'DONATION_UPDATE' | 'ALLOCATION_REQUEST' | 'DELIVERY_DISPATCHED' | 'SECURITY_ALERT'`), `title`, `body`, `actionUrl`, `isRead` (Boolean, default: false), `createdAt` (TTL index 30 days).
    - Indexes: `{ recipientId: 1, isRead: 1 }`, `{ createdAt: 1 }` (TTL).
15. **`EmailVerification`**
    - Fields: `_id`, `userId` (`Ref: User`), `tokenHash` (String), `expiresAt` (Date, TTL index), `attempts` (Number).
    - Indexes: `{ tokenHash: 1 }`, `{ expiresAt: 1 }` (TTL).
16. **`OTPVerification`**
    - Fields: `_id`, `phone` (String), `hashedOtp` (String), `purpose` (`'REGISTRATION' | 'LOGIN' | 'PASSWORD_RESET' | 'DELIVERY_HANDOVER'`), `expiresAt` (Date, TTL 5 mins), `attempts` (Number, max 3).
    - Indexes: `{ phone: 1, purpose: 1 }`, `{ expiresAt: 1 }` (TTL).
17. **`PasswordReset`**
    - Fields: `_id`, `userId` (`Ref: User`), `tokenHash` (String), `expiresAt` (Date, TTL 15 mins), `consumed` (Boolean, default false).
    - Indexes: `{ tokenHash: 1 }`, `{ expiresAt: 1 }` (TTL).
18. **`AuditLog`** & **`AIAction`**

- `AuditLog`: `_id`, `actorId` (`Ref: User`), `action` (String), `resourceType` (String), `resourceId` (String), `ipAddress`, `userAgent`, `previousState` (Object), `newState` (Object), `timestamp`.
- `AIAction`: `_id`, `invokedByAdminId` (`Ref: User`), `conversationId`, `toolName`, `toolInput` (Object), `toolOutput` (Object), `humanApproved` (Boolean), `executedAt`.

4. API Architecture (RESTful Endpoints)

All endpoints prefixed with `/api/v1`. Structured standard JSON response format:

```
json
{
  "success": true,
  "data": { ... },
  "message": "Human readable confirmation",
  "meta": { "timestamp": "2026-11-04T11:04:42.000Z", "traceId": "uuid" }
}
```

Route Groups & Permissions:

Route Path	Method	Access Role	Description
`/auth/register`	POST	Public	Register user & profile with Zod schema validation
`/auth/verify-otp`	POST	Public	Complete phone/email verification
`/auth/login`	POST	Public	Authenticate, issue HttpOnly JWT cookies
`/auth/refresh-token`	POST	Public (Cookie)	Rotate JWT using secure refresh token cookie
`/auth/logout`	POST	Authenticated	Clear session cookies, invalidate refresh token
`/auth/forgot-password`	POST	Public	Rate-limited password reset trigger
`/auth/reset-password`	POST	Public	Complete password reset with one-time token
`/donations`	POST	DONOR	Submit new food surplus donation
`/donations/my`	GET	DONOR	List personal donations with real-time status
`/donations/:id/cancel`	PATCH	DONOR, ADMIN	Cancel donation before dispatch
`/admin/donations/pending`	GET	ADMIN	View all unallocated verified donations
`/admin/match/:donationId`	GET	ADMIN	Trigger deterministic NGO matching engine
`/admin/allocate`	POST	ADMIN	Human approval for NGO allocation
`/ngo/requests`	GET	NGO_MANAGER	View incoming donation allocations
`/ngo/requests/:id/respond`	PATCH	NGO_MANAGER	Accept or reject donation allocation
`/logistics/assign-vehicle`	POST	ADMIN	Dispatch authorized vehicle/driver
`/logistics/delivery/:id/otp`	POST	NGO, DRIVER	Verify pickup or drop-off delivery OTP
`/logistics/delivery/:id/proof`	POST	NGO_MANAGER	Upload proof photo and close delivery
`/payments/create-order`	POST	DONOR	Initialize Razorpay monetary donation
`/payments/verify`	POST	DONOR	Webhook/client payment signature verification
`/ai/chat`	POST	ADMIN	Restricted AI operations assistant interaction
`/ai/approve-tool-action`	POST	ADMIN	Confirm and execute high-privilege AI recommendation

5. Authentication & Authorization Design

5.1 Token Strategy

- **Access Token:** Short-lived JWT (15 minutes), containing `{ sub: userId, role: role, exp: ... }`. Stored in `HttpOnly`, `SameSite=Strict`, `Secure` cookie named `fb_access`.
- **Refresh Token:** Long-lived cryptographically random 64-byte token (7 days), stored in `HttpOnly`, `SameSite=Strict`, `Secure`, `Path=/api/v1/auth/refresh-token` cookie named `fb_refresh`. Hashed with SHA-256 in MongoDB. Revoked on logout or reuse detection.

5.2 Password Security

- Hashed using **Argon2id** (memory cost: 65536 KB, iterations: 3, parallelism: 4) or Bcrypt with cost factor 12.

5.3 OTP Security

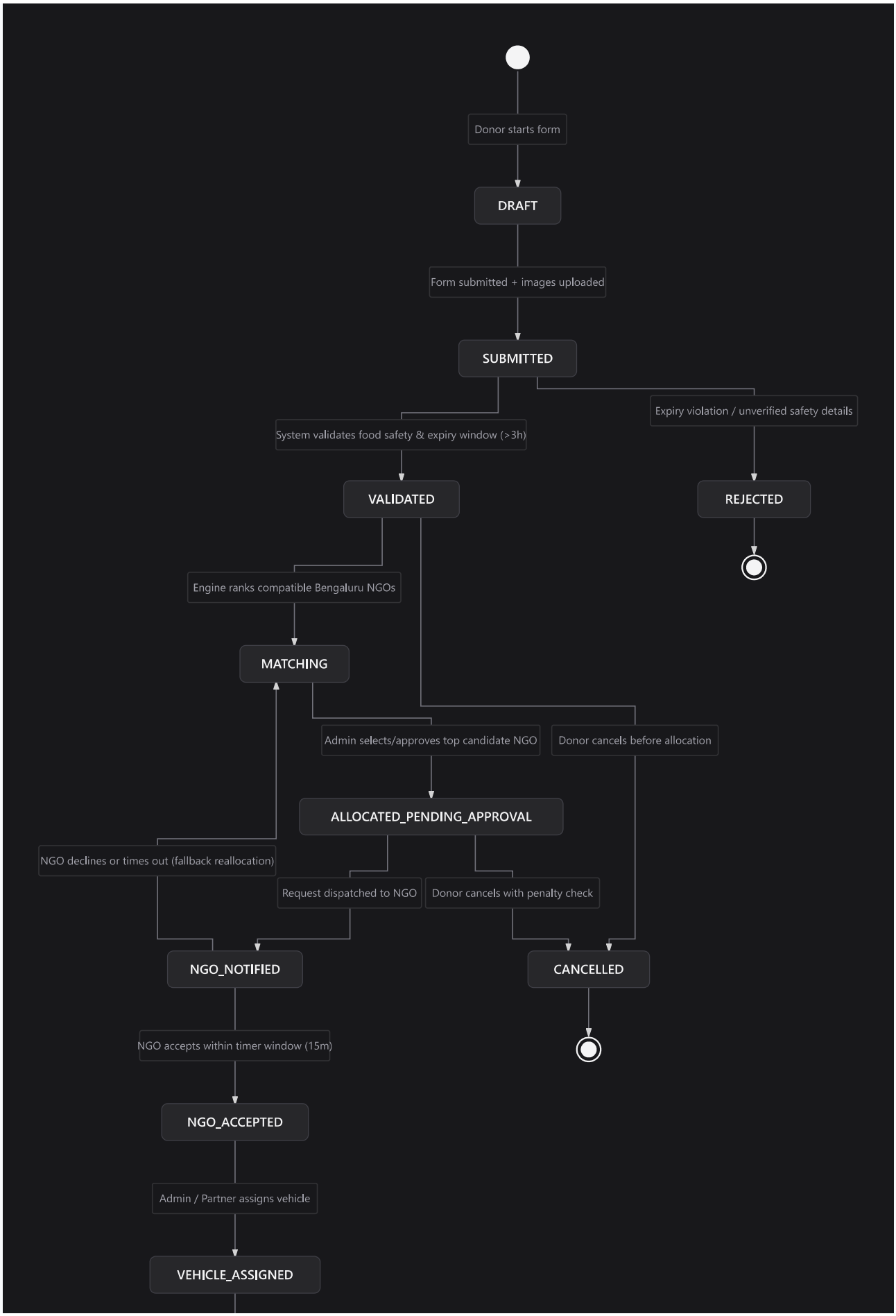
- Hashed using HMAC-SHA256 with an ephemeral server pepper before saving to `OTPVerification`.
- Rate limited: Max 3 attempts per OTP. Max 1 OTP generation per 60 seconds per phone.
- Expiry: 5 minutes.

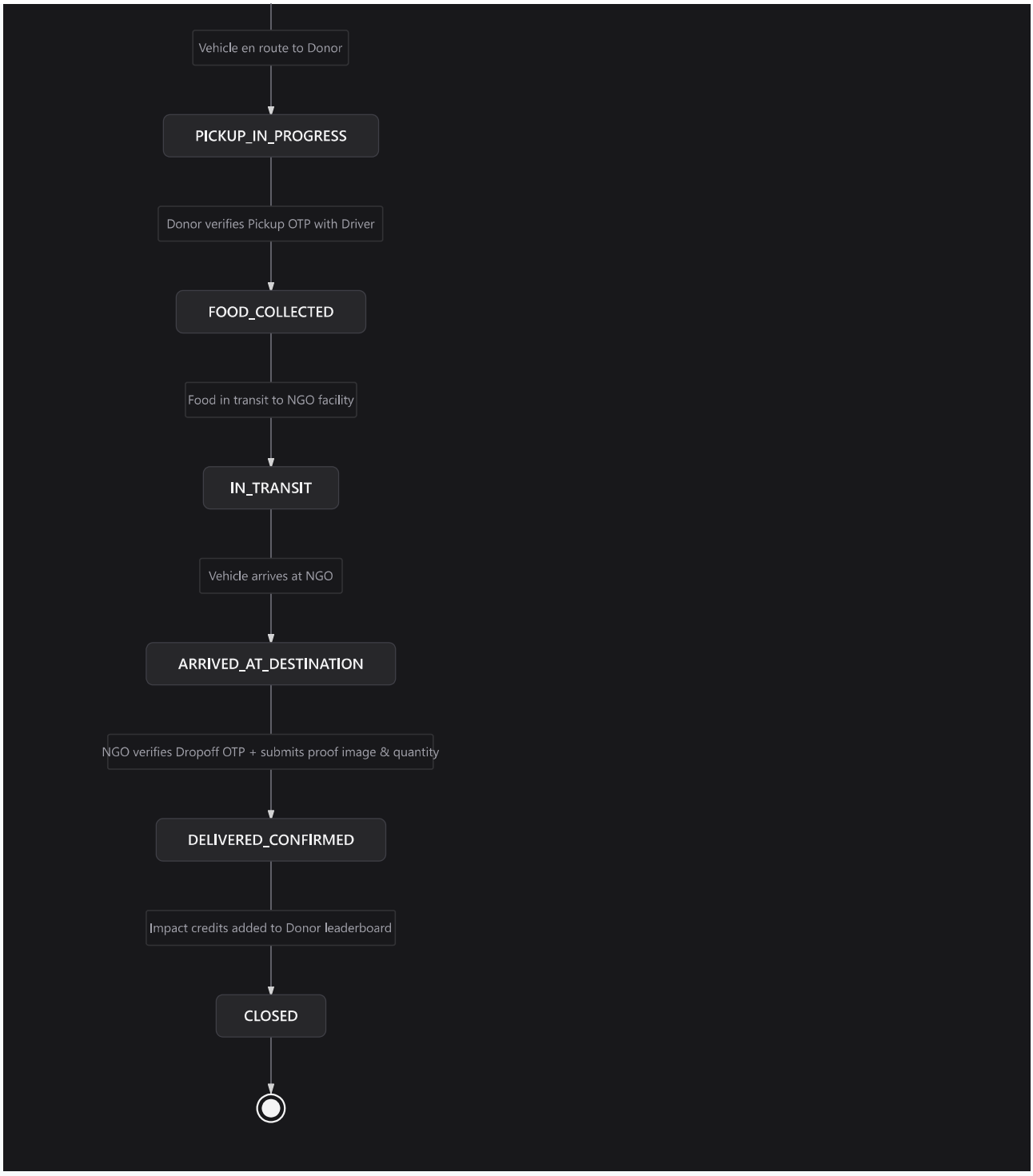
5.4 RBAC & Object-Level Access Control (ABAC)

- Express middleware chain: `authenticate` → → `requireRole(['ADMIN', 'NGO_MANAGER'])` → → `authorizeResourceOwnership(DonationModel, 'donorId')`.
- Ensures donors can never inspect or alter other donors' food listings or read aggregate beneficiary metrics belonging to NGOs.

6. Donation Lifecycle State Machine

To prevent double-allocations, race conditions, and deadlocks, all status changes pass through an atomic state machine:


Donor starts form
DRAFT
Form submitted + images uploaded
SUBMITTED
Expiry violation / unverified safety details
System validates food safety & expiry window (>3h)
VALIDATED
REJECTED
Engine ranks compatible Bengaluru NGOs
MATCHING
Admin selects/approves top candidate NGO
Donor cancels before allocation
ALLOCATED_PENDING_APPROVAL
NGO declines or times out (fallback reallocation)
Request dispatched to NGO
Donor cancels with penalty check
NGO_NOTIFIED
CANCELLED
NGO accepts within timer window (15m)
NGO_ACCEPTED
Admin / Partner assigns vehicle
VEHICLE_ASSIGNED



Concurrency & Invariant Enforcement

- MongoDB transactions (`session.withTransaction`) ensure an allocation is written with an atomic conditional lock: `findOneAndUpdate({ _id: donationId, status: 'VALIDATED' }, { $set: { status: 'ALLOCATING' } })`.
- If two admin requests arrive simultaneously, the second fails cleanly with a `409 Conflict`.

7. Deterministic AI Matching Algorithm Design

The matching engine is **100% deterministic, transparent, and auditable**. The LLM never invents scores or alters database state directly.

Formula:

$$\text{Matching Score } S = (w_1 \cdot C_{\text{dietary}}) + (w_2 \cdot C_{\text{capacity}}) + (w_3 \cdot P_{\text{distance}}) + (w_4 \cdot U_{\text{urgency}}) + (w_5 \cdot H_{\text{reliability}})$$

Matching Score S = (w1 · Cdietary) + (w2 · Ccapacity) + (w3 · Pdistance) + (w4 · Uurgency) + (w5 · Hreliability)

Where weights sum to 100 ($w_1 = 30, w_2 = 25, w_3 = 20, w_4 = 15, w_5 = 10$w1 = 30, w2 = 25, w3 = 20, w4 = 15, w5 = 10):

1. C_{dietary} **Cdietary (0 or 1):** Binary compatibility filter. Non-veg food is strictly rejected for pure-veg NGOs ($0$0). Matching gives $1.0$1.0.
2. C_{capacity} **Ccapacity (0 to 1):** Proportion of food quantity vs NGO's verified beneficiary headcount + remaining daily quota:
   $$C_{\text{capacity}} = \min(1.0, \frac{\text{NGO Headcount} \times \text{PortionFactor}}{\text{Donation Quantity}})$$
   Ccapacity = min (1.0, Donation QuantityNGO Headcount × PortionFactor)
3. P_{distance} **Pdistance (0 to 1):** Proximity decay over Bengaluru urban geography using Haversine / Google Distance Matrix:
   $$P_{\text{distance}} = \max(0, 1 - \frac{\text{Distance in km}}{25 \text{ km}})$$
   Pdistance = max (0, 1 − 25 kmDistance in km) *(Within 3 km = 1.0;* $> 25 \text{ km} = 0$> *25 km = 0 unless refrigerated transport is available).*
4. U_{urgency} **Uurgency (0 to 1):** Compares donation remaining shelf-life vs estimated transit time:
   $$U_{\text{urgency}} = 1 - \frac{\text{Estimated Transit Time}}{\text{Time to Expiry}}$$
   Uurgency = 1 − Time to ExpiryEstimated Transit Time
5. $H_{\text{reliability}}$ **Hreliability (0 to 1):** Historical acceptance and verified proof-of-receipt rate over the last 90 days.

8. AI Agent Tool Architecture (Restricted Operations Assistant)

The AI Operations Assistant uses Gemini with **strictly typed function calling** (Zod-backed JSON schemas). It is sandboxed to **read-only analytics** and **drafting recommendations**.

Tools Registry:

1. `getNearbyNGOs(lat: number, lng: number, radiusKm: number, dietaryType: string)` → → Reads MongoDB 2dsphere index.
2. `getNGORequirements(ngoId: string)` → → Returns current active headcount and storage constraints.
3. `getDonationDetails(donationId: string)` → → Fetches food type, quantity, expiry, donor pickup notes.

4. `rankNGOs(donationId: string)` → → Executes the deterministic mathematical ranking formula.
5. `suggestAllocation(donationId: string)` → → Formats top 3 NGO matches with deterministic rationale.
6. `calculateDeliveryRoute(origin: LatLng, destination: LatLng)` → → Queries Google Routes / OSRM API for transit duration and traffic alerts.
7. `getAvailableVehicles(capacityNeededKg: number, requiresRefrigeration: boolean)` → → Queries available fleet registry.
8. `generateOperationsReport(dateRange: { start: string, end: string })` → → Computes food rescued (kg), meals served, active alerts.

Human-in-the-Loop Constraint:

- The AI can **never** trigger: `approveAllocation`, `dispatchVehicle`, `processPayment`, or `modifyUserPermission`.
- When an admin prompts: "*Allocate donation #402 to Sunshine Shelter and book an auto*", the AI returns a structured recommendation card with a **"Click to Confirm & Execute"** button for the authenticated Admin.

9. Security Threat Model (OWASP Top 10 Baseline)

Threat / OWASP Category	Potential Vector in FoodBridge	Architecture Mitigation
A01: Broken Access Control	Donor viewing another donor's donation or NGO's internal beneficiary numbers	Strict middleware tenancy checking `donorId == req.user.id`; NGOs only see aggregate counts.
A02: Cryptographic Failures	Exposing API keys or storing passwords insecurely	Argon2id for passwords; zero hardcoded secrets (`dotenv` + Zod env schema); encrypted PII.
A03: Injection	MongoDB operator injection (`$gt: ""` in JSON query)	Mongoose strict typing; Zod request schema validation; sanitize input with `express-mongo-sanitize`.
A04: Insecure Design	Double-allocation of food leading to NGO starvation or waste	Atomic database state machine with ACID transactions; idempotency keys on allocation.
A05: Security Misconfiguration	CORS open to all domains, missing headers	Helmet.js enabled; CORS strictly locked to frontend origin; cookies `Secure`, `HttpOnly`, `SameSite=Strict`.
A06: Vulnerable Dependencies	Vulnerable NPM packages in supply chain	Weekly `npm audit`, Dependabot alerts, Semgrep static code analysis in GitHub Actions.
A07: Identification & Auth Failures	Brute forcing OTPs or login endpoints	Rate limiting with `express-rate-limit` (5 attempts / 15 mins); account lockout backoff.
A08: Software & Data Integrity	Tampering with proof of delivery photos or fake receipts	Cloudinary authenticated uploads via server-generated signatures; SHA-256 hash checks on receipt.
A09: Security Logging & Monitoring	Undetected unauthorized admin actions	Immutable `AuditLog` collection recording actor ID, IP, user-agent, prior state, new state.
A10: Server-Side Request Forgery	Attacker abusing route calculation to probe internal IPs	Whitelist external API hosts (Google APIs, Cloudinary); reject user-controlled fetch URLs.

10. UI/UX & Frontend Architecture

Brand Aesthetics:

- **Palette:** Forest Green (`#164B38`), Deep Green (`#103B2D`), Cream (`#FAF7EF`), Sage (`#DDE9DC`), Warm Orange (`#F28B45`), Charcoal (`#202820`).
- **Typography:** *Playfair Display* for storytelling headlines, *Inter* for legible data dashboards.

Public Experience (Storytelling & 3D):

- **Hero Section:** Smooth interactive 3D globe / meal vessel (React Three Fiber + Drei) showcasing real-time rescued meals counter.
- **Scroll-triggered Narrative (GSAP ScrollTrigger):**
    1. *Food Preparation & Surplus:* Kitchen steam, banquet surplus.
    2. *Quality Packaging:* Sealing, safety labeling.
    3. *Transit Route:* Simulated Bengaluru map route from Indiranagar to shelter in Whitefield.
    4. *Distribution & Joy:* Beneficiary meal delivery.
- *Performance Guardrail:* 3D renders on high-performance canvases with WebGL fallback for low-power mobile devices. Form pages and dashboard layouts contain **zero** CPU-heavy 3D loops to maintain accessibility and form responsiveness.

Portals:

- **Donor Portal:** Quick 3-step donation form, photo dropzone, live status stepper, 80G tax receipt download.
- **NGO Portal:** Large tap targets, real-time audio/visual alert for urgent incoming donations, 1-click accept/decline.
- **Admin Portal:** Live map of Bengaluru showing active donations, fleet pins, NGO clusters, AI assistant drawer.

11. Testing & Quality Assurance Strategy

1. **Unit Testing (Vitest):**
    - Deterministic matching algorithm score calculations with edge cases (distance limits, dietary clashes, capacity overflows).
    - State machine transition matrix validation (testing all invalid transitions throw explicit 400 errors).
    - Zod schema validations with malformed inputs.
2. **Integration Testing (Supertest + In-Memory MongoDB):**
    - Full auth flows (registration, OTP verification, cookie issuance, token rotation).
    - Complete donation creation → → admin allocation → → NGO acceptance → → OTP verification flow.
3. **End-to-End Testing (Playwright):**
    - Donor login → → create donation → → admin dashboard verification.
4. **Security Testing:**
    - Semgrep SAST scans.

- OWASP ZAP baseline scan against preview deployment.

12. Deployment Architecture

- **Frontend:** Vercel with automatic edge routing, asset caching, and gzip/brotli compression.
- **Backend:** Render (Web Service) running Node.js in production mode with clustering, graceful termination signals (`SIGTERM`), and health checks (`/api/v1/health`).
- **Database:** MongoDB Atlas M0/M10 cluster located in `ap-south-1` (Mumbai) for sub-15ms latency to Bengaluru.
- **Static Assets:** Cloudinary CDN for auto-compressing and watermarking food delivery proof images.

13. Phased Implementation Roadmap

```
⚠️ Failed to render Mermaid diagram: Invalid mermaid header: "gantt". Expected "graph TD", "flowchart LR", "stateDiagram-v2", etc.. Supported types: flowchart/graph, stateDiagram-v2, sequenceDiagram, classDiagram, erDiagram, xychart-beta, pie.
gantt
    title FoodBridge Phased Roadmap
    dateFormat  YYYY-MM-DD
    section Phase 1: Foundation & Schemas
    Monorepo setup & Shared Zod schemas     :p1_1, 2026-11-05, 2d
    Express server & 18 Mongoose models      :p1_2, after p1_1, 3d
    Auth system & Argon2id/OTP cookies      :p1_3, after p1_2, 2d
    section Phase 2: State Machine & Engine
    Donation State Machine implementation   :p2_1, after p1_3, 3d
    Deterministic Matching Algorithm & Tests :p2_2, after p2_1, 2d
    Logistics & OTP handover workflows      :p2_3, after p2_2, 3d
    section Phase 3: AI Assistant & Portals
    Gemini Restricted Tool Agent            :p3_1, after p2_3, 3d
    React Frontend & Tailwind UI Shell      :p3_2, after p3_1, 3d
    Donor, NGO & Admin Portals              :p3_3, after p3_2, 4d
    section Phase 4: 3D Story & Production
    R3F / Three.js 3D Hero & Narrative      :p4_1, after p3_3, 3d
    E2E Testing, Audits & Deployment        :p4_2, after p4_1, 2d
```



Module Specifications Summary

Module Name	Purpose	Primary Inputs	Primary Outputs	Key Dependencies	Security Controls	Acceptance Criteria
Auth & Identity	Secure credential & OTP authentication	User credentials, Phone OTP, Refresh token	HttpOnly JWT cookies, verified user session	Argon2id, crypto, Resend/SMS	Rate limiting, brute-force lockout, token rotation	Pass unit/integration auth test suite
Donation State Engine	Manages donation lifecycle with zero race conditions	Food details, expiry, donor coordinates	Validated donation record, state change events	Mongoose, MongoDB transactions	Strict role checks, ownership validation	Reject any invalid status jump with 400
Matching Algorithm	Identifies & ranks optimal Bengaluru NGOs	Donation payload, NGO location/capacity index	Ranked NGO candidate array with score breakdowns	Geospatial queries, distance calculation	Pure function, no external side effects	100% deterministic score for identical inputs
Logistics & Handover	Coordinates vehicle dispatch and physical delivery	Vehicle ID, pickup/drop coordinates, OTPs	Real-time delivery tracking, verified proof record	Google Maps / OSRM, Cloudinary	6-digit OTP verification at both ends	Proof image required to mark closed
AI Assistant	Provides admin operational insights & drafts allocations	Admin natural language queries	Tool execution results, human approval cards	Gemini API, restricted tools registry	Read-only tools, mandatory human-in-the-loop	Zero unauthorized DB writes from LLM